# What's an API?

**API Communication Methods**

**1. REST (Representational State Transfer)**

**2. SOAP (Simple Object Access Protocol)** →

SOAP is an older API communication method that **relies on XML-based messaging**.

Unlike REST, which is lightweight, SOAP is more structured and secure, making it ideal for banking, healthcare, and enterprise applications.

## 3. GraphQL

Unlike REST, which requires multiple API calls to fetch related data, GraphQL can **fetch all necessary data in a single request**.

**Example:** Fetching a user's profile and their recent posts in a single request.

## 4. gRPC

gRPC (Google Remote Procedure Call) is a **high-performance API communication method** that uses **Protocol Buffers (Protobuf)** instead of JSON or XML, making it faster and more efficient.

gRPC uses **binary data format** instead of text-based formats, reducing payload size and it supports **bidirectional streaming**, meaning the client and server can send data at the same time.

---

## Public API directories:

- RapidAPI – A marketplace for APIs with free & paid options.
- Postman API Network – A collection of public APIs.

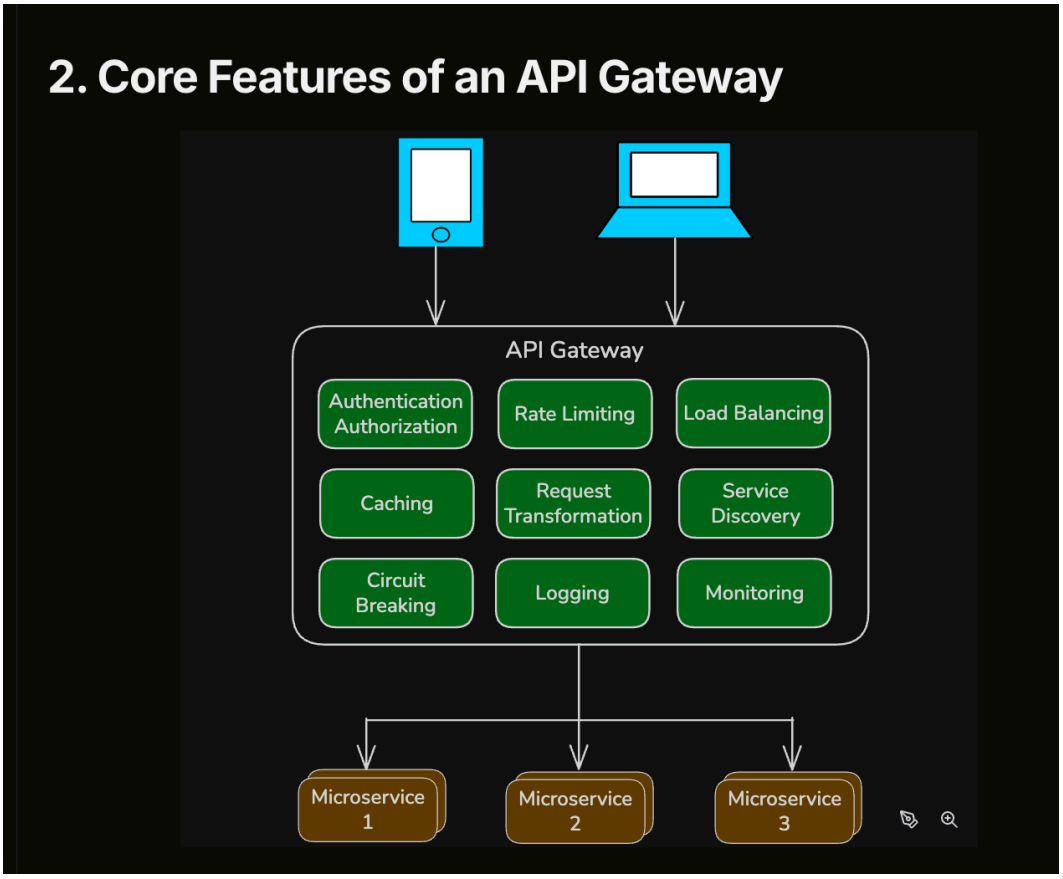
- API List – A fun list of free public APIs.
- GitHub's Public API List – Open-source API collection.

---

# API Gateway

As applications grow in size, the number of APIs increases too. Without the right tools and infrastructure, managing these APIs can quickly become a challenge.

This is where **API Gateway** comes into play




## 4: Rate Limiting

To prevent abuse, the API Gateway checks how many requests you've made recently. For example:

- If you've made 10 "Place Order" requests in the last minute (maybe by accident), the gateway might block additional requests temporarily and

return `429 Too Many Requests` response.

### 5. Request Transformation

- Modify the structure or format of incoming requests to match the backend service requirements.
- Transform responses before sending them back to the client, ensuring they meet the client's expectations.

### 6. Service Discovery

The **service discovery** feature of an API Gateway dynamically identifies the appropriate backend service instance to handle each request.

### 7. Circuit Breaking

Circuit breaking is a mechanism that temporarily stops sending requests to a backend service when it detects persistent failures, such as:

- Slow responses or timeouts.
- Server errors (e.g., HTTP 500 status codes).
- High latency or unavailability of a service

---

# REST vs GraphQL

## Benefits of REST

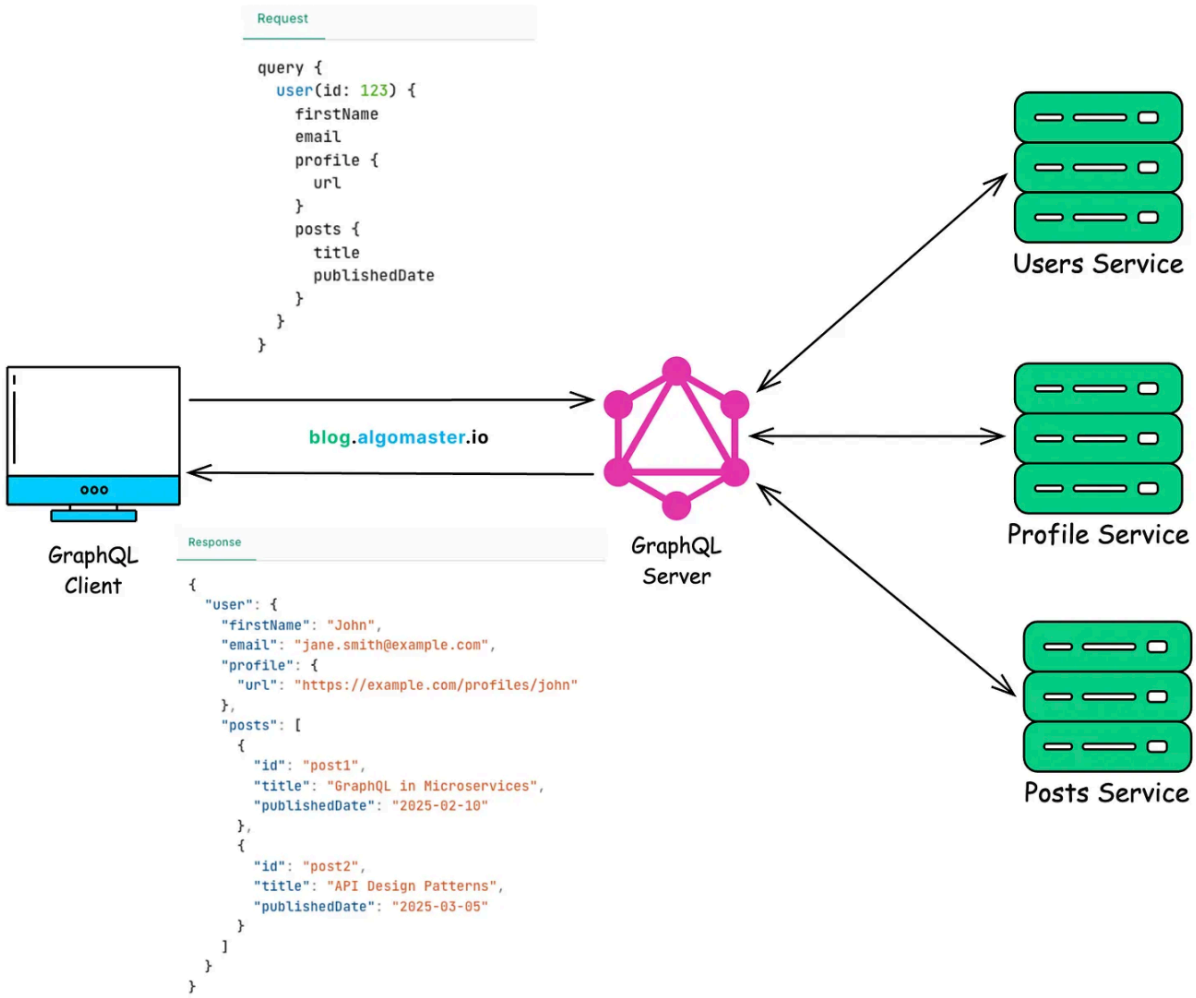
- **Simplicity and Intuitive Design**: The resource-based model aligns well with most business domains, making REST intuitive for developers.
- **Statelessness**: Each request contains all the information needed to complete it, making REST scalable across distributed systems.
- **Cacheability**: HTTP's caching mechanisms can be leveraged to improve performance.
- **Scalability:** REST APIs can be easily scaled using load balancers and CDNs.
- **Mature Ecosystem**: With nearly two decades of widespread use, REST enjoys robust tooling, documentation, and developer familiarity.

## Drawbacks of REST

- **Over-fetching**: REST endpoints often return **more data than needed**, leading to inefficient network usage. For example, if a mobile app only needs a user's name and email, but the API response includes additional fields like address, phone number, and metadata, it results in **wasted bandwidth**.
- **Under-fetching**: If an API doesn't return related data, the client may need to **make multiple requests** to retrieve all required information. For example, to get user details and their posts, a client might have to make:
  1. `GET /api/users/123` (fetch user)
  2. `GET /api/users/123/posts` (fetch user's posts)
- **Versioning issues**: When APIs evolve, maintaining backward compatibility becomes difficult. REST APIs often require **versioned URLs** (`/v1/users`, `/v2/users`), adding maintenance overhead.
- **Rigid Response Structure:** The server defines how data is returned, and clients must adapt to it—even if they only need a subset of the data.

# GRAPHQL

## Example:


Request
query {
user(id: 123) {
firstName
email
profile {
url
}
posts {
title
publishedDate
}
}
}
blog.algomaster.io
GraphQL Client
GraphQL Server
Users Service
Profile Service
Posts Service
Response
{
"user": {
"firstName": "John",
"email": "jane.smith@example.com",
"profile": {
"url": "https://example.com/profiles/john"
},
"posts": [
{
"id": "post1",
"title": "GraphQL in Microservices",
"publishedDate": "2025-02-10"
},
{
"id": "post2",
"title": "API Design Patterns",
"publishedDate": "2025-03-05"
}
]
}
}

GraphQL provides three core functionalities:

## 1. Queries → Fetch Data

Similar to GET requests in REST, GraphQL queries allow clients to request specific fields of data.

Clients have full control over what they retrieve, avoiding unnecessary data fetching.

**Example: Fetching specific user and post details in a single request**

```
query {
  user(id: 123) {
    name
    email
    posts {
      title
      content
    }
  }
}
```

## 2. Mutations → Modify Data

Equivalent to **POST, PUT, PATCH, or DELETE** in REST. Used to **create, update, or delete** resources in the API.

**Example: Creating a new post**

```
mutation {
  createPost(title: "GraphQL vs REST", content: "GraphQL solves many of REST's
limitations...", publishedDate: "2025-03-10") {
    id
    title
    content
  }
}
```

The response will contain the newly created post with its **ID, title, and content**.

### 3. Subscriptions → Real-Time Updates

Unlike REST, which requires polling or WebSockets for real-time updates, GraphQL subscriptions enable clients to listen for changes and receive updates automatically when data is modified.

Ideal for chat applications, live feeds, stock market updates, and notifications.

**Example: Listening for new posts**

```
subscription {
  newPost {
    title
    content
    author {
      name
    }
  }
}
```

Whenever a **new post is created**, all subscribed clients will **receive instant updates**.

## Benefits of GraphQL

1. **Precise Data Fetching**: Clients can request only the fields they need, reducing over-fetching and under-fetching.
2. **Single Request for Multiple Resources**: Related data can be retrieved in one request, solving REST's `n+1` query problem.
3. **Strong Typing**: GraphQL APIs use a schema to define available data, making them easier to explore and document.
4. **Real-time Data with Subscriptions:** GraphQL natively supports real-time data updates through subscriptions, enabling clients to receive automatic notifications whenever data changes on the server.
5. **API Evolution Without Versioning**: New fields can be added without breaking existing queries, avoiding REST-style `/v1`, `/v2` versioning issues.

## Drawbacks of GraphQL

1. **Complex Setup & Tooling**: Unlike REST, which can be used with basic HTTP clients (cURL, browsers), GraphQL requires a GraphQL server, schema, and resolvers.
2. **Caching challenges**: REST APIs leverage HTTP caching (e.g., browser caching, CDNs), but GraphQL queries use POST requests, making caching trickier.
3. **Increased Server Load:** Since clients can request arbitrary amounts of data, GraphQL APIs must be carefully optimized to prevent performance issues.
4. **Security Risks:** Unoptimized queries (e.g., deeply nested requests) can lead to costly database scans, increasing the risk of denial-of-service (DoS) attacks.

# WebSockets

The WebSocket connection starts with a standard HTTP request from the client to the server.

However, instead of completing the request and closing the connection, the server responds with an **HTTP 101** status code, indicating that the protocol is switching to WebSockets.

- **Bi-Directional**: Unlike HTTP, polling, and long-polling, WebSockets allow for two-way communication.
- **Low Latency**: Because the connection remains open, data can be sent and received with minimal delay.

**Summary Comparison**

| Feature | Long-Polling | WebSockets |
|---|---|---|
| **Connection Lifespan** | Opens and closes repeatedly. | Stays open permanently. |
| **Handshake Overhead** | High (TLS/TCP handshakes repeatedly). | Low (One handshake at the start). |
| **Data Overhead** | High (Heavy HTTP headers every time). | Very Low (2-10 byte frames). |
| **Authentication** | Re-authenticates on every new poll. | Authenticates once. |
| **Primary Bottleneck** | CPU and Network Bandwidth. | Server Memory (RAM). |

Export to Sheets

**The Catch:** You are absolutely right that WebSockets aren't "free." If you have a million active WebSocket connections, the server must keep a million file descriptors open in its RAM. If you run out of memory, the server will still crash. However, holding a connection in memory (WebSocket) is mathematically much cheaper than constantly rebuilding connections and parsing HTTP headers (Long-polling).

## 4. Challenges and Considerations

While WebSockets offer numerous benefits, there are some challenges to consider:

- **Proxy Servers**: Some proxy servers don't support WebSocket connections and certain firewalls may block them.
- **Scalability Concerns**: Managing a large number of WebSocket connections can be challenging. Consider using **load balancers** and distributed WebSocket servers to handle large-scale deployments.
- **Fallback Mechanism**: Not all clients or networks support WebSockets, which can lead to connectivity issues. Implement fallback mechanisms like long-polling for clients that cannot establish WebSocket connections.
- **Network Reliability**: WebSockets rely on a persistent connection, which can be disrupted by network issues. Implementing **reconnection strategies** and **heartbeat mechanisms** (regular ping/pong messages) can help maintain the connection's stability and detect when a connection has been lost.
- **Security**: WebSockets are vulnerable to attacks such as Cross-Site WebSocket Hijacking and Distributed Denial of Service (DDoS) attacks. Implement **secure WebSocket connections** (wss://), authenticate users, and validate input to protect against common vulnerabilities.

### 1. Real-Time Collaboration Tools

Applications like Google Docs may use WebSockets to enable multiple users to edit a document simultaneously. Changes made by one user are instantly reflected for all others, creating a seamless collaborative experience.

### 2. Real-Time Chat Applications

One of the most popular uses of WebSockets is in real-time chat applications.

Messaging platforms like Slack use WebSockets to deliver messages instantly. This allows for real-time conversations and immediate message delivery notifications.

### 3. Live Notifications

Social media platforms use WebSockets to push real-time notifications to users when they receive a new message, like, or comment.

Instead of the client constantly checking for new notifications, the server can push updates to the client as soon as they occur.

### 4. Multiplayer Online Games

In online multiplayer games, low latency is crucial for a seamless gaming experience.

WebSockets provide the necessary real-time communication between the game server and players, ensuring that all players see the same game state simultaneously.

### 5. Financial Market Data Feeds

WebSockets are widely used in financial applications to stream real-time market data, such as stock prices, forex rates, and cryptocurrency values.

### 6. IoT (Internet of Things) Applications

In IoT applications, devices often need to communicate with a server in real time.

WebSockets provide a lightweight and efficient communication channel for sending sensor data, receiving commands, and synchronizing device states.

### 7. Live Streaming and Broadcasting

While the actual video streaming typically uses other protocols, WebSockets can be used for real-time chat, viewer counts, and other interactive features during live broadcasts.

# Idempotency

Imagine you're making a **purchase** from an online store.You hit **"pay"** but the screen freezes, and you're unsure if the payment went through.

So, you **refresh** the page and **try again**.Behind the scenes, how does the system ensure you aren’t accidentally **charged twice**?

an operation is idempotent if applying it multiple times produces the same result as applying it once.

# 1. Unique Request Identifiers

One of the simplest techniques to achieve idempotency is by attaching a **unique identifier**, often called an **idempotency key** to each request.

> **Example**: A payment service could require every transaction request to include a unique ID. If the client retries with the same ID, the server will skip the charge, preventing duplicate transactions.

## 3. Challenges and Considerations

While idempotency is powerful, it comes with its own set of challenges:

1. **Performance Overhead**: Storing idempotency keys or checking for duplicate operations can add overhead and increase the overall latency.
2. **State Management**: Idempotency often requires maintaining state, which can be challenging in stateless architectures.
3. **Distributed Systems**: Ensuring idempotency across distributed systems can be challenging and may require **distributed locking** or **consensus algorithms**.
4. **Time Window**: How long should idempotency guarantees be maintained? Forever, or for a limited time?
5. **Database Constraints**: Not all operations are idempotent by default; unique constraints or upsert logic may be necessary to avoid duplication.

---

## 4. Best Practices

When implementing idempotency in your system, consider these best practices:

1. **Use Unique Identifiers**: Attach a unique ID (idempotency key) to each request to track and prevent duplicate processing.
2. **Design for Idempotency from the Start**: It's much easier to design for idempotency from the beginning than to add it later.
3. **Implement Retry with Backoff**: When retrying idempotent operations, use an exponential backoff strategy to avoid overwhelming the system.
4. **Employ Idempotent HTTP Methods**: Prefer idempotent methods (GET, PUT, DELETE) for operations that may be retried; design POST with unique identifiers if idempotency is required.
5. **Document Idempotent Operations**: Clearly document which operations are idempotent in your API specifications.
6. **Test Thoroughly**: Implement tests that verify the idempotency of your operations, including edge cases and failure scenarios.
7. **Use Locks or Versioning**: Use locks, optimistic concurrency control, or version numbers to manage simultaneous requests safely.

# Rate Limiting Algorithms

Rate limiting helps protects services from being overwhelmed by too many requests from a single user or client.

## 1. Token Bucket

- **The Concept:** You have a bucket that holds a set number of tokens. A new token is added at a constant rate (e.g., 1 per second). Every time a user makes an API request, it "spends" one token. If the bucket is empty, the request is blocked.
- **The Con Explained (The "Burst" Vulnerability):** Because the bucket stores up tokens, it allows for a "burst" of traffic. If you set the bucket size to 10,000 to allow for high traffic, a user could theoretically send 10,000 requests in a single millisecond. The algorithm will happily allow all of them through because the tokens are available. If your backend database can only handle 2,000 operations per second, that sudden burst will crash your server.

## 2. Leaky Bucket

- **The Concept:**
- **The Queue (The Bucket):** You give a user a fixed-size queue. Let's say the queue can hold exactly 5 requests.
- **The Constant Processing (The Leak):** Your server promises to process the requests in that queue at a strict, unchangeable speed. For example: exactly 1 request every second. No faster, no slower.
- **Handling a Burst:** If a user suddenly clicks a button 5 times in half a second, all 5 requests are accepted and placed into the queue.
- **The Catch (The Drop):** Because the server is strictly processing them at 1 per second, the queue is now full and will take 5 seconds to empty. If the user tries to send a 6th request right now, the queue is full. That 6th request "spills over" and is instantly rejected.
- **The Con Explained (Hostage to Old Requests):** The Leaky Bucket strictly forbids bursts. If a sudden surge of traffic comes in, the bucket fills up instantly. Because it only processes requests at a rigid, slow pace, those

requests just sit there. If a brand new, highly critical request comes in a few seconds later, the bucket is still full of the older, slower requests—meaning the new critical request is instantly dropped. It essentially penalizes new traffic when a traffic spike happens.

## 3. Fixed Window Counter

- **The Concept:** Time is sliced into rigid windows (e.g., exactly 1:00 PM to 1:01 PM). You are allowed a fixed number of requests per window. At the start of the new minute, the counter resets to zero.
- **The Con Explained (The Boundary/Edge Flaw):** This is the most famous flaw in rate limiting. Imagine your limit is 100 requests per minute.
    - At **1:00:55**, a user sends 100 requests. The counter hits 100, and they are all accepted.
    - At **1:01:00**, the clock ticks over. The counter resets to 0.
    - At **1:01:05**, the user sends another 100 requests. The counter hits 100, and they are all accepted.
    - **The Problem:** The user just successfully sent **200 requests within a 10-second window** (between 1:00:55 and 1:01:05). By timing their traffic right at the edge of the resetting clock, they completely bypassed your intended server limits.

## 4. Sliding Window Log (The Perfectionist)

- **How it works:** It literally writes down the exact timestamp of *every single request* you make. If you make 100 requests, it saves 100 timestamps (e.g., `[1:01:05, 1:01:08, 1:01:12...]`). When a new request comes in, it looks at the exact last 60 seconds, deletes any timestamps older than that, and counts what is left.
- **The Good:** It is **100% accurate**. It perfectly prevents bursts at any given millisecond.
- **The Bad:** It is a **massive memory hog**. Storing millions of timestamps for millions of users in a fast database like Redis is incredibly expensive and slows down the server.

### 5. Sliding Window Counter (The Pragmatist)

- **How it works:** It refuses to save individual timestamps. Instead, it only saves **two numbers** per user: the total count of the previous minute, and the total count of the current minute. It then uses the math formula (overlap percentage) we discussed above to guess your current rate.

1. **The Formula:** `Current Window Requests` + `(Previous Window Requests × Overlap Percentage)`

### A Simple Example

Imagine it is exactly **1:01:15 PM** (You are 15 seconds, or **25%**, into the current minute).

- **Previous Minute (1:00 - 1:01):** You made **80** requests.
- **Current Minute (1:01 - 1:02):** You have made **10** requests so far.

Because you are 25% into the new minute, a true 60-second "sliding" window looking backward would cover 25% of the current minute, and **75%** of the previous minute.

So, the algorithm calculates your estimated current traffic:

- Take 75% of the previous window: `80 requests × 0.75 = 60 requests`
- Add the current window: `10 requests`
- **Your Estimated Rate:** `60 + 10 = 70 requests`

Since 70 is less than your limit of 100, your new request is **Accepted**.

- **The Good:** It uses **almost zero memory**. Storing two simple numbers (e.g., `Prev: 80, Curr: 10`) takes up practically no space, making it lightning-fast and cheap to run at scale.
- **The Bad:** It is **not perfectly accurate**. Because it uses math to estimate your traffic based on an "even distribution" assumption, it might occasionally let a tiny burst slip through.

---

# Webhook

A webhook is a mechanism that allows one system to send real-time data to another system as soon as an event occurs, using an HTTP request. Instead of continuously asking a service if an event has happened (a resource-heavy process called **polling**), webhooks allow the provider to push the information directly to you.

- *Analogy:* Instead of constantly asking a restaurant host if your table is ready, you give them your number so they can text you when it is.

Suppose you make a course , now you want if someone buys course an email is sent to them

or

suppose your github code is deployed on vercel , now you want whenever code gets pushed on github , vercel api is called

So instead vercel continuously polling github if new code has come , github sends alert to vercel only when code is pushed

So in this case there is endpoint of vercel such as /api/vercel/webhook

this api is registered in github , whenever code gets pushed , it send alerts to all webhooks attached with it.

# 4.1 Basic Setup

## Create a Dedicated Endpoint

Start by exposing a simple HTTP endpoint like:

`POST /webhook`

This endpoint should:

- Accept only POST requests
- Accept only JSON (via `Content-Type`)
- Be accessible over HTTPS

### Make It Idempotent

Webhook events can be **retried**, **duplicated**, or even **replayed** by provid

### Verify the Signature (HMAC)

Most providers include a **cryptographic signature** of the payload, using a shared secret. This ensures the payload came from the real source.

## Designing a Scalable Webhook Infrastructure

# 5.1 Queue Incoming Requests

Don't do heavy processing inside your webhook endpoint.

Instead:

- Parse and validate the request
- Verify its authenticity (signature, event ID, etc.)
- Immediately enqueue the event into a **message queue** like:
- **Kafka** (for high-throughput event pipelines)
- **RabbitMQ** (lightweight and flexible)
- **AWS SQS** (serverless and easy to manage)

# 5.3 Process with Asynchronous Workers

Background workers (or worker pools) pull events from the queue and perform actual business logic:

- Deduplicate the event using eventID
- Run validations
- Update internal databases or call downstream services
- Trigger notifications or workflows

**Why use workers?**

- You can scale them horizontally

- You gain control over load and processing rate
- You isolate failures from the webhook endpoint

## 5.4 Retry with Backoff

Sometimes, event processing fails temporarily due to a database issue, timeout, or an unavailable downstream API.

Your workers should **retry** those events automatically, using a **backoff strategy**:

- **Exponential** (recommended): retry with increasing delays (1s, 2s, 4s...)

## 5.5 Use a Dead Letter Queue (DLQ)

If a webhook consistently fails after multiple retries, don't keep retrying forever.

Instead, send it to a **Dead Letter Queue**, a special holding queue for problematic events.

From there, you can:

- Trigger alerts
- Investigate the root cause